Topological-Temporal Neural Networks

Elvin Isufi

Intelligent Systems Department

May 15, 2026

Topological Linear Models

- Topological-VAR incorporates the topology into VAR model
  - $\Rightarrow$ Linear input-output model
  - $\Rightarrow$ focuses predominantly on low-level information – noise in the time series data

Topological Linear Models

- Topological-VAR incorporates the topology into VAR model
  - $\Rightarrow$ Linear input-output model
  - $\Rightarrow$ focuses predominantly on low-level information – noise in the time series data

- Topological state-space models account for the latent structure
  - $\Rightarrow$ Linear in the input-output models
  - $\Rightarrow$ grows the focus on the information present in the latent

Topological Linear Models

- Topological-VAR incorporates the topology into VAR model
  - ⇒ Linear input-output model
  - ⇒ focuses predominantly on low-level information – noise in the time series data

- Topological state-space models account for the latent structure
  - ⇒ Linear in the input-output models
  - ⇒ grows the focus on the information present in the latent

- RNNs (GRUs,LSTMs) are the non-linear NN extension of VAR
  - ⇒ Goal: Incorporate topological structure into these models

Outline: What will you learn today?

- How to learn representations from time-varying data over topologies?

  $\Rightarrow$ Topologically-aware disjoint models

  $\Rightarrow$ Topologically-aware recursive models

Topologically-aware Disjoint Models

Breaking Down Topological-Temporal Models

Topological Component:

Models relations among simplices through the complex

SCNN | SCCNN | Hodge filters | Dirac filters

Temporal Component:

Models sequential dependencies in the time series on the simplices

CNN | RNN | LSTM | Attention

Topological-Temporal Interaction determines type of model

- **Disjoint:** topology and time are processed in separate blocks, without explicitly coupling topological propagation and temporal memory.
- **Joint:** topology and time are fused, capturing topological-temporal interactions, e.g., topological recursive models or product-space models.

Topological Time Series

- Input: a collection of topological time series

$$\left\{\mathbf{X}^{(k)} \in \mathbb{R}^{N_k \times T}\right\}_{k=0}^{K}$$

- It can be divided into topological signals or time series

  $\Rightarrow$ Topological signals: a column of data

$$\mathbf{x}_t^{(k)} \in \mathbb{R}^{N_k}, \qquad t \in \{1, \ldots, T\}, \quad k \in \{0, \ldots, K\}$$

  $\Rightarrow$ Time series: a row of data

$$\mathbf{x}_{i,:}^{(k)} = \left[x_{i1}^{(k)}, \ldots, x_{iT}^{(k)}\right], \qquad i \in \{1, \ldots, N_k\}$$

- Here, k denotes the simplex order:

$$k = 0: \text{ nodes}, \qquad k = 1: \text{ edges}, \qquad k = 2: \text{ triangles}.$$

Disjoint Models: General Pipeline

- Apply disjointly topological learning and time learning.

- For each time t, the topological encoder processes the multi-order signal

$$\left\{\mathbf{x}_t^{(k)}\right\}_{k=0}^K \quad \longmapsto \quad \left\{\mathbf{z}_t^{(k)}\right\}_{k=0}^K .$$

- Then, the outputs across simplex orders are aggregated into a single feature vector

$$\mathbf{z}_t = \mathrm{stack}\left(\mathbf{z}_t^{(0)}, \mathbf{z}_t^{(1)}, \ldots, \mathbf{z}_t^{(K)}\right).$$

- Finally, a temporal model processes the embedding sequence

$$\widehat{\mathbf{y}}_t = \Phi_{\mathrm{time}}\left(\mathbf{z}_1, \ldots, \mathbf{z}_t\right).$$

Disjoint Models: Topological Layer

- Topological layer: for $t \in \{1, \dots, T\}$

$$\text{embedding} \longleftarrow \mathbf{z}_t = \Phi_{\text{topo}}\left(\{\mathbf{x}_t^{(k)}\}_{k=0}^{K}; \mathcal{K}\right) \longleftarrow \text{multi-order signal}$$

Topological model

- The output $\mathbf{z}_t$ is then treated as a temporal feature vector
- Topological model can be a topological filter, SCNN, SCCNN, or other topological encoder

Disjoint Models: Temporal Layer

- Temporal layer: process the embedding sequence $\mathbf{z}_1, \ldots, \mathbf{z}_T$

$$\text{hidden state} \longleftarrow \mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{z}_t; \mathcal{H}) \longrightarrow \text{Learnable parameters}$$
$$\uparrow$$
$$\text{Embedding/time series}$$

- The output is then obtained from the hidden state, e.g.,

$$\widehat{\mathbf{y}}_t = \rho(\mathbf{h}_t).$$

- $f(\cdot)$ can be any temporal model: RNN/LSTM/GRU

SCRNN: A Disjoint Topological-Temporal Model

- Instantiate the previous pipeline with the Simplicial Convolutional Recurrent Neural Network

  spike trains $\longrightarrow$ simplicial complex $\longrightarrow$ SC layers $\longrightarrow$ RNN $\longrightarrow$ decoded variable.

- The goal is neural decoding:
  - $\Rightarrow$ head direction from head-direction cell activity
  - $\Rightarrow$ animal location from grid-cell activity
- SCRNN is a topological encoder followed by a temporal model.

Mitchell et al., "A topological deep learning framework for neural spike decoding", 2024.

SCRNN Pipeline

- SCRNN follows the disjoint topology-time pipeline:

$$\mathbf{A} \longrightarrow \mathcal{K} \longrightarrow \mathbf{z}_t \longrightarrow \mathbf{h}_t \longrightarrow \widehat{\mathbf{y}}_t.$$

- $\mathbf{A}$: spike-count matrix; $\mathcal{K}$: simplicial complex built from neural co-activity; $\mathbf{z}_t$: feature vector produced by SC layers; $\mathbf{h}_t$: hidden state of the RNN.
- The topology is used to build features before the RNN.

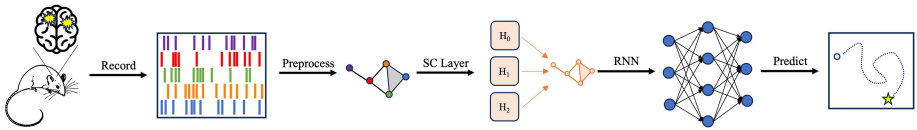


Fig. credit from Mitchell et al

From Spikes to a Simplicial Complex

- Start from binned spike activity:

$$\mathbf{A} \in \mathbb{R}^{N \times N_b}.$$

  Rows correspond to neurons; columns correspond to time bins.

- The spike matrix is thresholded:

$$\mathbf{A} \longrightarrow \bar{\mathbf{A}}.$$

  Simultaneously active neurons form simplices:

$$n_{\text{act}} \text{ active neurons} \longrightarrow (n_{\text{act}} - 1)\text{-simplex}.$$

- The simplicial complex captures multi-neuron co-activity, not only pairwise relations.

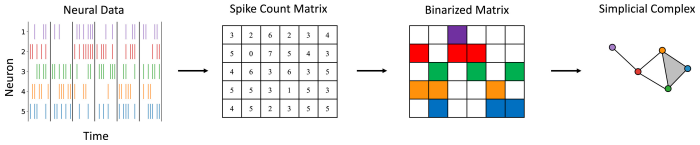


Fig. credit from Mitchell et al

SCRNN: SC Layers Followed by RNN

- Topological part: each simplicial complex is processed by SC layers

$$\mathcal{K} \longrightarrow \text{SC layers} \longrightarrow \mathbf{z}_t.$$

- The SC-layer outputs are flattened or concatenated into one feature vector:

$$\mathbf{z}_t = \text{stack}\left(\mathbf{z}_t^{(0)}, \ldots, \mathbf{z}_t^{(K)}\right).$$

- Temporal part: consecutive feature vectors form the RNN input sequence

$$\mathbf{z}_1, \ldots, \mathbf{z}_T \longrightarrow \text{RNN} \longrightarrow \widehat{\mathbf{y}}_t.$$

- In the RNN used in SCRNN:

$$\mathbf{h}_t = \sigma\left(\mathbf{W}_h \mathbf{z}_t + \mathbf{b}_h + \mathbf{W}_c \mathbf{h}_{t-1} + \mathbf{b}_c\right), \qquad \widehat{\mathbf{y}}_t = \rho(\mathbf{W}\mathbf{h}_t + \mathbf{b}).$$

SCRNN Architecture

- The SC layers extract topology-aware features from each time bin. The output of the final SC layer is flattened into a vector $\mathbf{z}_t$.
- The RNN receives a sequence of these vectors:

$$\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_T.$$

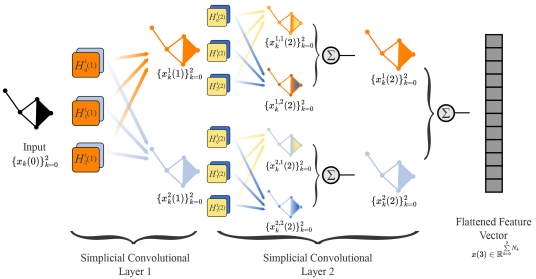


Fig. credit from Mitchell et al

SCRNN Numerical Results

Question: does higher-order topology help neural decoding?

- Tasks: head-direction decoding and grid-cell location decoding.
- Models: FFNN, RNN, GNN, SCRNN
- Metrics: AAE is average absolute angular error in degrees; AED is average Euclidean distance in cm.

Model	**Head direction** AAE in degrees	**Grid cells** AED in cm
FFNN	17.990	11.801
RNN	14.587	3.350
GNN	14.624	3.539
SCRNN	11.493	3.088

- Take-home: SCRNN outperforms GNNs, suggesting that higher-order simplicial connectivity is useful for sequential decoding

Disjoint Models: Pros and Cons

Pros

- Everything is ready: easier and efficient to implement
- Modular design:
  - $\Rightarrow$ any topological encoder
  - $\Rightarrow$ any temporal model
- Useful as a strong baseline
- Can show that topology helps the task

Cons

- Does not capture topological-temporal dependencies inside the recurrence
- Does not allow topology to structure latent memory dynamics
- Difficult to isolate whether the gain comes from topology or the temporal model
- Limited theory for the joint topological-temporal mechanism

- Bridge: if topology helps the features, can it also help the memory update?

Topologically-aware Recursive Models

Models Overview

Topological recursive models

- Inject topology in temporal memory methods
- Capture joint topological-temporal interactions in the data

Topological recursive models
Topology-aware memory models

- Low complexity
- Scalable
- parameters topology-independent

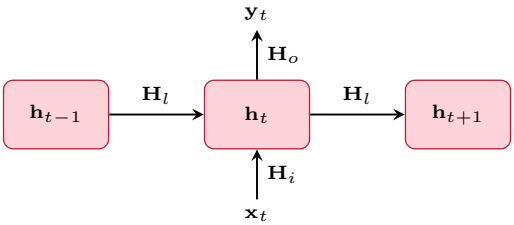
$$\mathbf{h}_t = \rho\left(\mathbf{H}_l \mathbf{h}_{t-1} + \mathbf{H}_i \mathbf{x}_t\right)$$
$$\Downarrow$$
$$\mathbf{h}_t = \rho\left(\Phi_l(\mathcal{K}) \mathbf{h}_{t-1} + \Phi_i(\mathcal{K}) \mathbf{x}_t\right)$$

- State-space models
- AR models
- RNNs, GRU, LSTM

- Main idea: substitute model matrices by topological filters/operators!

Recurrent Neural Networks (RNN) Model

$$\mathbf{h}_t = \rho(\mathbf{H}_l \mathbf{h}_{t-1} + \mathbf{H}_i \mathbf{x}_t)$$
$$\mathbf{y}_t = \rho(\mathbf{H}_o \mathbf{h}_t)$$



- Parameter matrices $\mathbf{H}_l, \mathbf{H}_i, \mathbf{H}_o \in \mathbb{R}^{N \times N}$

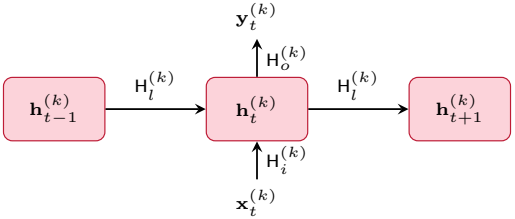
State can also be of dimension $d < N$

- Main idea: an RNN updates a hidden state recursively through time.

Topological RNNs

Substitute parameter matrices $\mathbf{H}$ with topological filters $\mathsf{H}^{(k)}(\mathbf{L}_k^{\downarrow}, \mathbf{L}_k^{\uparrow})$

$$\mathbf{h}_t^{(k)} = \rho\left(\mathsf{H}_l^{(k)}(\mathbf{L}_k^{\downarrow}, \mathbf{L}_k^{\uparrow})\mathbf{h}_{t-1}^{(k)} + \mathsf{H}_i^{(k)}(\mathbf{L}_k^{\downarrow}, \mathbf{L}_k^{\uparrow})\mathbf{x}_t^{(k)}\right)$$
$$\mathbf{y}_t^{(k)} = \rho\left(\mathsf{H}_o^{(k)}(\mathbf{L}_k^{\downarrow}, \mathbf{L}_k^{\uparrow})\mathbf{h}_t^{(k)}\right)$$

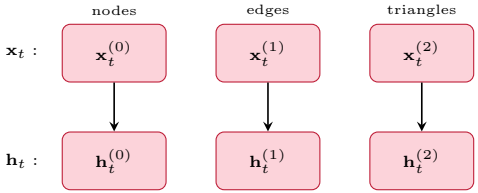


- Learn now filter coefficients

$$\{h_{0r}^{(k)}\}, \{h_{\downarrow,pr}^{(k)}\}, \{h_{\uparrow,pr}^{(k)}\}, \qquad r \in \{l, i, o\}.$$

Topological RNNs: Multi-Order State

- At each time t, the input is a collection of simplex signals $\left\{\mathbf{x}_t^{(k)} \in \mathbb{R}^{N_k}\right\}_{k=0}^{K}$.
- A topological RNN maintains a hidden state on each simplex order $\left\{\mathbf{h}_t^{(k)} \in \mathbb{R}^{N_k \times d}\right\}_{k=0}^{K}$.
- The recurrent state is therefore itself a topological signal $\mathbf{h}_t^{(0)}, \mathbf{h}_t^{(1)}, \ldots, \mathbf{h}_t^{(K)}$.



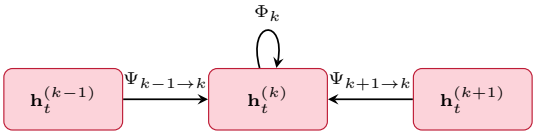
- Main idea: the memory is distributed over the topology.

Topological RNNs: Joint Update

- The order-wise update can be enriched by exchanging information across simplex orders.

$$\mathbf{h}_t^{(k)} = \rho\left(\text{same-order information} + \text{lower-order information} + \text{higher-order information}\right).$$

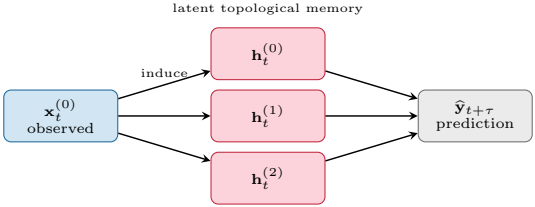
- More explicitly, one can write $\mathbf{h}_t^{(k)} = \rho(\Phi_k + \Psi_{k-1\to k} + \Psi_{k+1\to k})$.
- Here:
  - $\Rightarrow$ Φ_k: information propagated within k-simplices
  - $\Rightarrow$ $\Psi_{k-1\to k}$: information injected from lower-order simplices
  - $\Rightarrow$ $\Psi_{k+1\to k}$: information injected from higher-order simplices



- Main idea: topology is now part of the recurrent update, not only of the input features.

Higher-Order Structure May Be Latent

- In many applications, the observed signal is only on nodes $\mathbf{y}_t = \mathbf{x}_t^{(0)}$.
- However, the hidden memory can still live on all simplex orders $\mathbf{h}_t^{(0)}, \mathbf{h}_t^{(1)}, \ldots, \mathbf{h}_t^{(K)}$.



- Higher-order latent space can be induced by the observed signals and then processed with higher-order topological filters.
- Examples: traffic speeds induce latent flows/congestion patterns; neural spikes induce latent co-activity motifs.
- Main idea: higher-order topology can structure memory even when the prediction target is node-level.

What Can Topological RNNs Be Used For?

Tasks

- Forecasting

$$\{\mathbf{x}_1, \ldots, \mathbf{x}_t\} \longrightarrow \widehat{\mathbf{x}}_{t+\tau}$$

- Imputation / reconstruction of missing topological signals
- Anomaly detection in dynamical systems over complexes
- Classification of temporal regimes or trajectories

Why joint recurrence?

- Memory is explicitly topology-aware

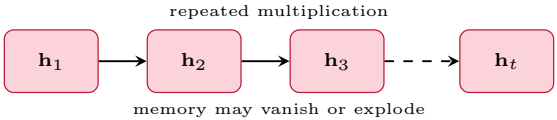
$$\mathbf{h}_t = \Phi_{\text{top-rec}}\left(\mathbf{h}_{t-1}, \mathbf{x}_t; \mathcal{K}\right)$$

- Less information is lost between topology and time modules
- Higher-order latent states can influence the temporal update
- Topological properties can extend to the recurrent model: equivariance, stability

Why Vanilla RNNs Are Not Enough

- Vanilla RNNs update the memory with the same recursive rule

$$\mathbf{h}_t = \rho\left(\mathbf{H}_l \mathbf{h}_{t-1} + \mathbf{H}_i \mathbf{x}_t\right).$$



- This can make long-range dependencies hard to learn.
- There is no explicit mechanism deciding

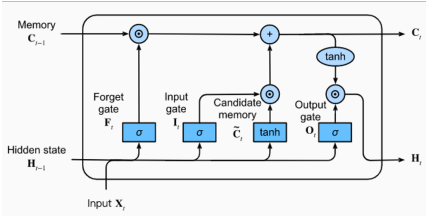
what to write, what to forget, what to output.

- The same issue remains for topological RNNs if the recurrent cell is still too simple.

LSTM: Gated Memory

- LSTMs introduce an explicit cell state $\mathbf{c}_t$ and gates controlling the memory.

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \widetilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$



- Gate roles:
  - $\Rightarrow$ $\mathbf{f}_t$: forget gate, controls how much previous memory is retained
  - $\Rightarrow$ $\mathbf{i}_t$: input gate, controls how much of the candidate memory is written
  - $\Rightarrow$ $\widetilde{\mathbf{c}}_t$: candidate memory update
  - $\Rightarrow$ $\mathbf{o}_t$: output gate, controls how much memory is exposed as hidden state

xLSTM: Exponential Gating

- xLSTM revisits LSTMs with two main changes:
  - $\Rightarrow$ exponential gates with normalization/stabilization
  - $\Rightarrow$ modified memory structures: sLSTM and mLSTM
- In the scalar-memory sLSTM variant, the input and forget gates use exponential activations:

$$\tilde{\mathbf{i}}_t, \tilde{\mathbf{f}}_t = \text{gate preactivations}, \qquad \mathbf{i}_t \propto \exp(\tilde{\mathbf{i}}_t), \qquad \mathbf{f}_t \propto \exp(\tilde{\mathbf{f}}_t).$$

- The memory update keeps the LSTM form

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{z}_t,$$

  where $\mathbf{z}_t$ is the cell input/candidate update.
- Why exponential gates?
  - $\Rightarrow$ They are positive and not bounded by 1, unlike sigmoid gates
  - $\Rightarrow$ They allow stronger write/overwrite decisions
  - $\Rightarrow$ Normalization/stabilization is then used to keep the recurrence controlled

STORM: Topological Recurrent Model

Simplicial Topological Recurrent Model

- STORM combines two ideas:
  - $\Rightarrow$ gated memory from LSTM/xLSTM-type recurrent cells
  - $\Rightarrow$ topological filters inside the recurrent gates

topological time series $\longrightarrow$ topology-aware gates $\longrightarrow$ topological memory $\longrightarrow$ forecast

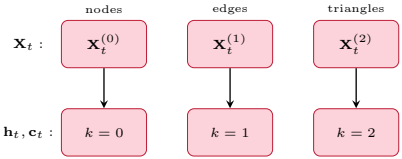
- Unlike disjoint models, topology is not used only to build features before recurrence.
- In STORM, the topology controls

what is written, what is forgotten, what is read out.

- Main idea: STORM puts simplicial filters inside the gates of a recurrent memory model.

STORM States

- STORM is defined on a fixed simplicial complex $\mathcal{K}$.
- At time t, each order k has an input signal $\mathbf{X}_t^{(k)} \in \mathbb{R}^{N_k \times d_k}$, $k = 0, \ldots, K$.
- For each order, STORM maintains hidden and cell states $\mathbf{h}_t^{(k)}, \mathbf{c}_t^{(k)} \in \mathbb{R}^{N_k \times H}$.
- The recurrent memory is therefore multi-order: $\left\{\mathbf{h}_t^{(k)}, \mathbf{c}_t^{(k)}\right\}_{k=0}^{K}$.



- Main idea: STORM stores memory on nodes, edges, and higher-order simplices.

STORM Recurrent Update

- For each order k, STORM uses input, candidate, output, and timescale pre-activations

$$\mathbf{G}_i^{(k)}, \qquad \mathbf{G}_z^{(k)}, \qquad \mathbf{G}_o^{(k)}, \qquad \mathbf{G}_\tau^{(k)}.$$

- Gates and candidate memory:

$$\mathbf{i}_t^{(k)} = \sigma(\mathbf{G}_i^{(k)}), \qquad \widetilde{\mathbf{c}}_t^{(k)} = \tanh(\mathbf{G}_z^{(k)}), \qquad \mathbf{o}_t^{(k)} = \sigma(\mathbf{G}_o^{(k)}).$$

- Exponential forget gate with positive timescale:

$$\boldsymbol{\tau}_t^{(k)} = \mathrm{softplus}(\mathbf{G}_\tau^{(k)}), \qquad [\mathbf{f}_t^{(k)}]_{ij} = \exp\left(-\Delta t / [\boldsymbol{\tau}_t^{(k)}]_{ij}\right).$$

- Cell and hidden states:

$$\mathbf{c}_t^{(k)} = \mathbf{f}_t^{(k)} \odot \mathbf{c}_{t-1}^{(k)} + \mathbf{i}_t^{(k)} \odot \widetilde{\mathbf{c}}_t^{(k)}, \qquad \mathbf{h}_t^{(k)} = \mathbf{o}_t^{(k)} \odot \tanh(\mathbf{c}_t^{(k)}).$$

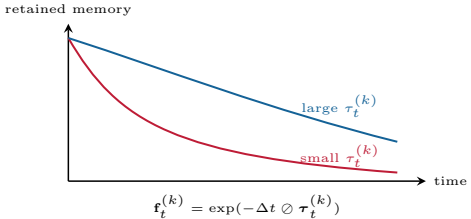
- Main idea: each simplex and hidden channel has its own adaptive memory decay.

STORM: Exponential Forget Gate

- STORM uses a positive, topology-dependent timescale for the forget gate.

$$\boldsymbol{\tau}_t^{(k)} = \mathrm{softplus}\left(\mathbf{G}_\tau^{(k)}\right), \qquad \mathbf{f}_t^{(k)} = \exp\left(-\Delta t \oslash \boldsymbol{\tau}_t^{(k)}\right).$$

- The cell update remains LSTM-like: $\mathbf{c}_t^{(k)} = \mathbf{f}_t^{(k)} \odot \mathbf{c}_{t-1}^{(k)} + \mathbf{i}_t^{(k)} \odot \widetilde{\mathbf{c}}_t^{(k)}$.



- A small timescale forgets quickly; a large timescale preserves memory longer.
- Main idea: STORM learns how fast each topological memory component should decay.

STORM Gates Are Topological

- The pre-activation of each gate $g \in \{i, z, o, \tau\}$ is computed from two branches:

$$\mathbf{G}_g^{(k)} = \mathbf{G}_{X,g}^{(k)} + \mathbf{G}_{h,g}^{(k)} + \mathbf{1}_{N_k} (\mathbf{b}_g^{(k)})^\top .$$

- Input branch:

$$\mathbf{G}_{X,g}^{(k)} = \Phi_{X,g}^{(k)} \left(\{\mathbf{X}_t^{(j)}\}_{j=0}^{K}; \mathcal{K} \right) .$$

- Hidden branch:

$$\mathbf{G}_{h,g}^{(k)} = \Phi_{h,g}^{(k)} \left(\{\mathbf{h}_{t-1}^{(j)}\}_{j=0}^{K}; \mathcal{K} \right) .$$

- The functions $\Phi_{X,g}^{(k)}$ and $\Phi_{h,g}^{(k)}$ are simplicial convolutional filters with cross-order injections.
- Main idea: topology shapes the gates, hence it shapes memory writing, forgetting, and reading.

Same- and Cross-Order Topology

- For each gate and order, STORM combines two kinds of topological information.

$$\mathbf{G}_{X,g}^{(k)} = \underbrace{\text{intra-order filtering of } \mathbf{X}_t^{(k)}}_{\mathbf{L}_k^{\downarrow},\ \mathbf{L}_k^{\uparrow}} + \underbrace{\text{cross-order injection from } k-1, k+1}_{\mathbf{B}_k,\ \mathbf{B}_{k+1}}.$$

- In expanded form:

$$\mathbf{G}_{X,g}^{(k)} = \mathbf{X}_t^{(k)}\boldsymbol{\Theta}_{g,\mathrm{id}}^{(k)} + \sum_{p=1}^{P}(\mathbf{L}_k^{\downarrow})^p\mathbf{X}_t^{(k)}\boldsymbol{\Theta}_{g,\downarrow,p}^{(k)} + \sum_{p=1}^{P}(\mathbf{L}_k^{\uparrow})^p\mathbf{X}_t^{(k)}\boldsymbol{\Theta}_{g,\uparrow,p}^{(k)}$$
$$+ \sum_{p=0}^{P}(\mathbf{L}_k^{\downarrow})^p\widetilde{\mathbf{X}}_{t,\downarrow}^{(k)}\boldsymbol{\Theta}_{g,\mathrm{inj},\downarrow,p}^{(k)} + \sum_{p=0}^{P}(\mathbf{L}_k^{\uparrow})^p\widetilde{\mathbf{X}}_{t,\uparrow}^{(k)}\boldsymbol{\Theta}_{g,\mathrm{inj},\uparrow,p}^{(k)}$$

- Cross-order injected signals are obtained through the boundary maps:

$$\widetilde{\mathbf{X}}_{t,\downarrow}^{(k)} = \mathbf{B}_k^{\top}\mathbf{X}_t^{(k-1)}, \qquad \widetilde{\mathbf{X}}_{t,\uparrow}^{(k)} = \mathbf{B}_{k+1}\mathbf{X}_t^{(k+1)}.$$

- The hidden branch is analogous, replacing $\mathbf{X}_t$ by $\mathbf{h}_{t-1}$.

STORM Injection Modes

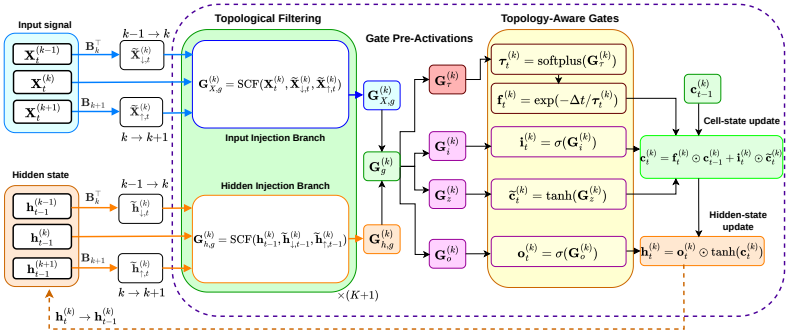
- STORM injection modes control where cross-order interactions occur, not whether a branch is used.
- In all modes, both branches contribute to each gate pre-activation:

$$\mathbf{G}_g^{(k)} = \mathbf{G}_{X,g}^{(k)} + \mathbf{G}_{h,g}^{(k)} + \mathbf{1}_{N_k} (\mathbf{b}_g^{(k)})^\top.$$

Variant	Input branch $\mathbf{X}_t$	Hidden branch $\mathbf{h}_{t-1}$
STORM-input	intra-order + cross-order	intra-order only
STORM-hidden	intra-order only	intra-order + cross-order
STORM-both	intra-order + cross-order	intra-order + cross-order

- Cross-order injection is switched on in the input branch, hidden branch, or both.

STORM Architecture



- Two parallel SCF branches process the current input $\mathbf{X}_t^{(k)}$ and previous hidden state $\mathbf{h}_{t-1}^{(k)}$.
- Cross-order signals from $k-1$ and $k+1$ are injected through boundary operators.
- The resulting gate pre-activations drive the recurrent update for $g \in \{i, z, o, \tau\}$.

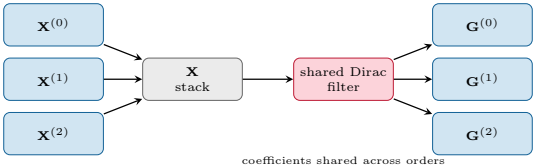
LightSTORM: Shared Dirac Filters

- LightSTORM stacks all orders and processes them with shared Dirac-polynomial filters.

$$\mathbf{X} = \text{stack}\left(\mathbf{X}^{(0)}, \ldots, \mathbf{X}^{(K)}\right) \in \mathbb{R}^{N \times H}, \qquad N = \sum_{k=0}^{K} N_k.$$

$$\mathbf{G}_{X,g} = \mathbf{X}\boldsymbol{\Theta}_{g,0} + \sum_{p=1}^{J} \left(\mathbf{D}^{(d)}\right)^p \mathbf{X}\boldsymbol{\Theta}_{g,p}^{(d)} + \sum_{p=1}^{J} \left(\mathbf{D}^{(u)}\right)^p \mathbf{X}\boldsymbol{\Theta}_{g,p}^{(u)}.$$

- The hidden branch is analogous, replacing $\mathbf{X}$ with $\mathbf{h}$.



Theoretical Properties

- Permutation equivariance: relabeling nodes, edges, and triangles should not change the model, except for the same relabeling of the states and outputs.
- Stability: small perturbations of the weighted topological operators should produce small changes in the predictions.
- Generalization: the model complexity should remain controlled even when processing long sequences.

Permutation Equivariance

- Let $\mathbf{P}_k \in \{0,1\}^{N_k \times N_k}$ be a permutation matrix relabeling the k-simplices.
- Relabel the complex and the input signals consistently:

$$\mathbf{B}'_k = \mathbf{P}_{k-1}\mathbf{B}_k\mathbf{P}_k^\top, \qquad \mathbf{X}_t^{(k)\prime} = \mathbf{P}_k\mathbf{X}_t^{(k)}.$$

Proposition: If STORM is runs on the relabeled complex with relabeled initial states, then for all $t \geq 0$ and $k = 0, \ldots, K$,

$$\mathbf{h}_t^{(k)\prime} = \mathbf{P}_k\mathbf{h}_t^{(k)}, \qquad \mathbf{c}_t^{(k)\prime} = \mathbf{P}_k\mathbf{c}_t^{(k)}.$$

- $\mathbf{h}_t^{(k)}$ and $\mathbf{c}_t^{(k)}$ are the hidden and cell states on k-simplices.
- Meaning: STORM depends on the topology, not on the arbitrary ordering of nodes, edges, or triangles.

Orientation Sensitivity

- Orientation: reversing orientations introduces sign flips in oriented signals and boundary matrices
- Hodge and boundary filters are orientation equivariant

Remark: STORM is not orientation equivariant, because the sigmoid, softplus, clamp, and exponential gate nonlinearities do not commute with sign flips.

- Odd nonlinearities are insufficient: input, forget, and output gates multiply sign-changing states, and the gate values would need to remain orientation invariant

Stability to Topology Perturbations

- $\mathcal{K}$ be the original complex and $\widehat{\mathcal{K}}$ a perturbed version
- The boundary operators are perturbed as

$$\widehat{\mathbf{B}}_k = \mathbf{B}_k + \Delta\mathbf{B}_k, \qquad \|\Delta\mathbf{B}_k\|_2 \leq \eta r_k.$$

- η is the perturbation size and r_k is the scale of the k-th boundary operator

Theorem: Under bounded-filter and contractive-memory conditions, running STORM on $\mathcal{K}$ and $\widehat{\mathcal{K}}$ yields constants $C_{\mathrm{BIBO}}, C'_{\mathrm{BIBO}} < \infty$ such that

$$\|\mathbf{Y}_t - \widehat{\mathbf{Y}}_t\| \leq C_{\mathrm{BIBO}}\eta + C'_{\mathrm{BIBO}}\eta^2, \qquad t \geq 1.$$

- Meaning: small topology perturbations induce controlled output changes, uniformly over time.

Generalization Bound

- Let $f_\theta \in \mathcal{F}_T$ be a STORM predictor trained on M trajectories $\left(\mathbf{X}_{1:T}^{(m)}, \mathbf{Y}^{*(m)}\right)$.

- $R(f_\theta)$ and $\widehat{R}_M(f_\theta)$ denote the true and empirical risks.

Theorem: If the recurrent state class contracts in Rademacher complexity with factor $q_{\text{gen}} < 1$, then with probability at least $1 - \delta$, uniformly over $\theta \in \Theta$,

$$R(f_\theta) \leq \widehat{R}_M(f_\theta) + \frac{2L_\ell c_{\text{out}} \mathfrak{C}_{\text{full}}}{(1 - q_{\text{gen}})\sqrt{M}} + B_\ell \sqrt{\frac{\log(2/\delta)}{2M}}.$$

$$\mathfrak{C}_{\text{full}} := C_{\text{gate}}(K+1)(4P+3)\left[W_{\max}\bar{r}(B_X + B_{\mathcal{S}}) + b_{\max}\right], \qquad \bar{r} := \max\left(1, \max_k r_{k,\downarrow}, \max_k r_{k,\uparrow}\right).$$

- L_ℓ, B_ℓ: loss Lipschitz constant and bound; c_{out}: readout Lipschitz constant; P: filter order; $K+1$: number of simplex orders; $W_{\max}, b_{\max}$: bounds on filter matrices and biases; $B_X, B_{\mathcal{S}}$: bounds on inputs and recurrent states; $\bar{r}$: bound on cross-order operator norms.

- C_{gate}: universal constant collecting the fixed Lipschitz constants of gate nonlinearities and Hadamard products.

Numerical Setup: Forecasting Task

- At time t, the model observes signals on multiple simplex orders and predicts

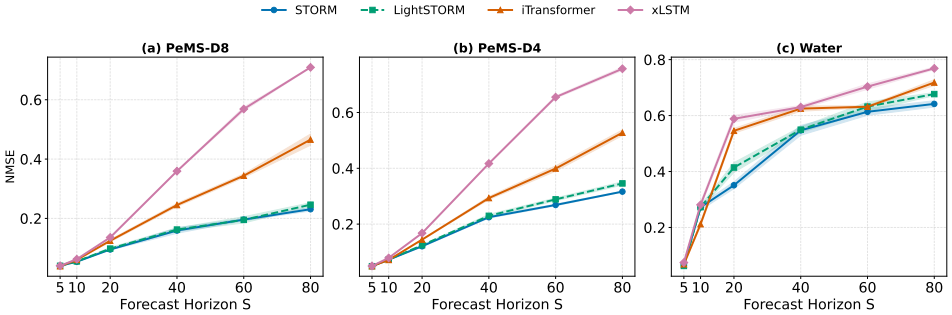
$$\widehat{\mathbf{X}}_{t+1:t+S}^{(k)} = \mathbf{h}_t^{(k)} \mathbf{W}_{\text{out}}^{(k)} + \mathbf{b}_{\text{out}}^{(k)}, \qquad k = 0, \ldots, K.$$

- The decoder outputs all S forecast steps in one forward pass, with $\in \{5, 10, 20, 40, 60, 80\}$

Dataset	T	N_0	N_1	N_2	**Signals**
PeMS-D4	16,992	307	340	29	node traffic; lifted edge/triangle signals
PeMS-D8	17,856	170	274	80	node traffic; lifted edge/triangle signals
Water	4,480	36	40	0	node pressure; edge flow

- Main idea: forecast node, edge, and triangle signals jointly whenever available.

Main Forecasting Results



NMSE vs. forecasting step S (mean ± 1 std over 5 seeds; lower is better). For STORM and LightSTORM, each point reports the injection variant with the lowest mean NMSE at the corresponding dataset and horizon.

- Take-home: gains are largest at medium and long horizons on the traffic networks and become consistent from medium horizons onward on Water.

Main Forecasting Results: Take-Home

- Long-horizon forecasting: STORM is most useful when the prediction horizon increases.
  - $\Rightarrow$ Gains are modest or mixed at short horizons
  - $\Rightarrow$ Improvements become clearer at medium and long horizons

Main Forecasting Results: Take-Home

- Long-horizon forecasting: STORM is most useful when the prediction horizon increases.
  - $\Rightarrow$ Gains are modest or mixed at short horizons
  - $\Rightarrow$ Improvements become clearer at medium and long horizons
- Topology helps memory:
  - $\Rightarrow$ topology-aware models improve over temporal and graph-based baselines
  - $\Rightarrow$ Using topology inside the recurrent state, not only as a preprocessing step

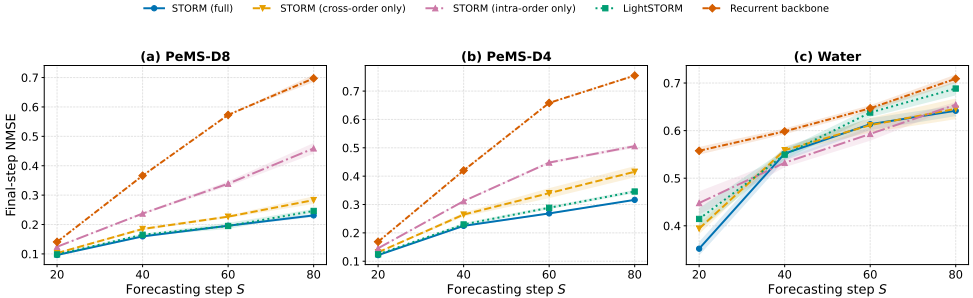
Main Forecasting Results: Take-Home

- Long-horizon forecasting: STORM is most useful when the prediction horizon increases.
  - $\Rightarrow$ Gains are modest or mixed at short horizons
  - $\Rightarrow$ Improvements become clearer at medium and long horizons
- Topology helps memory:
  - $\Rightarrow$ topology-aware models improve over temporal and graph-based baselines
  - $\Rightarrow$ Using topology inside the recurrent state, not only as a preprocessing step
- Richer topology helps more:
  - $\Rightarrow$ gains are strongest on traffic datasets with nodes, edges, and triangles.
  - $\Rightarrow$ Water: complex has nodes and edges but no triangles, improvements are present but less uniform

Main Forecasting Results: Take-Home

- Long-horizon forecasting: STORM is most useful when the prediction horizon increases.
  - $\Rightarrow$ Gains are modest or mixed at short horizons
  - $\Rightarrow$ Improvements become clearer at medium and long horizons
- Topology helps memory:
  - $\Rightarrow$ topology-aware models improve over temporal and graph-based baselines
  - $\Rightarrow$ Using topology inside the recurrent state, not only as a preprocessing step
- Richer topology helps more:
  - $\Rightarrow$ gains are strongest on traffic datasets with nodes, edges, and triangles.
  - $\Rightarrow$ Water: complex has nodes and edges but no triangles, improvements are present but less uniform
- Main message: STORM is most beneficial when forecasting requires both long-term temporal memory and higher-order topological structure.

Role of Topology



- Take-home: topology-aware variants separate more clearly from the topology-free recurrent backbone at longer horizons.

Per-Order Results: PeMS-D4

Model	S=40			S=80		
	Nodes	Edges	Triangles	Nodes	Edges	Triangles
STORM (both)	**0.2318**	**0.2200**	0.2030	**0.3278**	**0.3120**	0.2433
STORM (hidden)	0.2324	0.2228	**0.1930**	0.3380	0.3264	0.2535
LightSTORM (both)	0.2360	0.2265	0.1990	0.3560	0.3453	**0.2377**
LightSTORM (hidden)	0.2500	0.2432	0.2075	0.3769	0.3730	0.2803
iTransformer	0.2982	0.2878	0.3046	0.5307	0.5273	0.4904
xLSTM	0.4230	0.4105	0.4296	0.7504	0.7621	0.7615
GCN-xLSTM	0.3347	0.3840	0.4482	0.5863	0.6902	0.7532
SC-VAR	0.5827	0.6183	0.6822	0.9027	0.9619	0.9383

- Take-home: improvements are visible not only on nodes, but also on edge and triangle forecasts.

Per-Order Results: PeMS-D8

Model	S=40			S=80		
	Nodes	Edges	Triangles	Nodes	Edges	Triangles
STORM (both)	0.1600	0.1622	**0.1496**	**0.2272**	**0.2379**	**0.2153**
STORM (hidden)	**0.1598**	**0.1612**	0.1510	0.2316	0.2439	0.2244
LightSTORM (both)	0.1630	0.1686	0.1584	0.2352	0.2569	0.2244
LightSTORM (hidden)	0.1609	0.1659	0.1532	0.2438	0.2627	0.2265
iTransformer	0.2440	0.2460	0.2443	0.4488	0.4742	0.4718
xLSTM	0.3466	0.3646	0.3681	0.6621	0.7303	0.7374
GCN-xLSTM	0.2371	0.3104	0.3391	0.4453	0.6063	0.6627
SC-VAR	0.5220	0.5840	0.6080	0.7946	0.9392	0.9483

- Take-home: at $S = 80$, STORM gives the strongest per-order results across nodes, edges, and triangles.

Empirical Take-Home

What do the ablations add beyond the main tables?

- Topology matters: removing topological structure degrades forecasting, especially at longer horizons.
- Where topology enters matters: hidden and both injection modes are usually stronger than input-only injection.

  $\Rightarrow$ This suggests that higher-order coupling is especially useful inside the recurrent memory.
- Higher-order forecasting improves: gains are visible not only on node signals, but also on edge and triangle signals.
- The effect is strongest when the topology is richer, as in the traffic complexes with nodes, edges, and triangles.

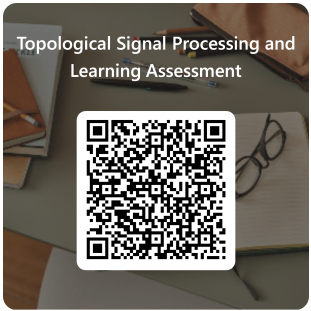
Learning Activity: Where Should Topology Enter the Memory?

Topologically-aware recursive models do not only process the current input with topological filters. In models such as STORM, topology can also shape the recurrent gates, deciding what is written, forgotten, and read out from memory across nodes, edges, and higher-order simplices.

- Q9.1: You are forecasting traffic over a simplicial complex with node speeds, edge flows, and triangle-level congestion patterns. Long-term prediction requires remembering how higher-order flow patterns evolve over time. Which design choice best matches the goal of a topologically-aware recurrent model?

- Instructions
  - ⇒ Scan the QR code to open the activity
  - ⇒ Work individually first
  - ⇒ Then discuss your answer in pairs or groups of 3
  - ⇒ Submit your final answer in Microsoft Forms

- Logistics
  - ⇒ Time: 10 min
  - ⇒ Goal: Understand why topology should structure recurrent memory, not only input features
  - ⇒ Product: One submitted response per student + justification to peers



https://forms.office.com/

Further reading

- Topological signal processing and learning
  - ⇒ Barbarossa and Sardellitti, Topological Signal Processing over Simplicial Complexes, 2020
  - ⇒ Yang et al., Simplicial Convolutional Filters, 2022
  - ⇒ Yang et al., Simplicial Convolutional Neural Networks, 2022
  - ⇒ Krishnan et al., Simplicial Vector Autoregressive Models, 2024
  - ⇒ Mitchell et al., A Topological Deep Learning Framework for Neural Spike Decoding, 2024
  - ⇒ Jebali et al., STORM: Simplicial Topological Recurrent Models, under review
- Recurrent and spatio-temporal models
  - ⇒ Hochreiter and Schmidhuber, Long Short-Term Memory, 1997
  - ⇒ Beck et al., xLSTM: Extended Long Short-Term Memory, 2024
  - ⇒ Liu et al., iTransformer, 2023
- Theory
  - ⇒ Gama et al., Stability Properties of Graph Neural Networks, 2020
  - ⇒ Bartlett and Mendelson, Rademacher and Gaussian Complexities, 2002
  - ⇒ Mohri et al., Foundations of Machine Learning, 2018